

Reinforcement Learning pentru conducere autonomă într-un mediu personalizat “Highway”

Echipa Juno
Corobana Ingrid
Moise Irina
Prizlopan Iustin
Sescu Matei

Ianuarie 2026

1 Formularea problemei

Obiectivul proiectului este antrenarea unui agent de control pentru un vehicul *ego* (mașina galbenă) într-un mediu de trafic. Agentul trebuie să circule pe un drum cu mai multe benzi, să evite coliziunile cu vehiculele din trafic (mașinile albastre), să rămână pe carosabil și, în același timp, să mențină o viteză rezonabilă.

Problema este formulată ca o problemă de **Reinforcement Learning (RL)**, în care agentul învață o politică (*policy*) exclusiv pe baza recompensei primite de la mediu.

1.1 Stare, acțiune, recompensă

Modelăm problema în termeni de *stare*, *acțiune* și *recompensă* astfel:

- **Stare (observație):** observația de bază a mediului este de tip *Kinematics* (din pachetul `highway_env`). După aplicarea `FlatObsWrapper`, obținem un vector continuu de dimensiune **25**, care conține poziția, viteza și orientarea vehiculului *ego*, plus informații agregate despre câteva vehicule din proximitate.
- **Acțiune:** mediul original are acțiuni continue (acelerație și direcție). Pentru agenții discreți folosim un wrapper `DiscreteActionWrapper` care definește **5 acțiuni**: *Idle* (0), *Accelerate* (1), *Brake* (2), *Left* (3), *Right* (4).
- **Recompensă:** funcția de recompensă din `custom_env.py` combină mai multe componente: penalizări puternice pentru **coliziune** sau **ieșirea de pe drum** și termeni pozitivi pentru **viteză ridicată** (aliniată cu sensul benzii) și pentru **menținerea poziției aproape de centrul benzii** (*lane centering*). În acest fel, agentul primește feedback la fiecare pas de timp, nu doar la finalul episodului.

Diagrama generală agent–mediu este prezentată în Figura 1.

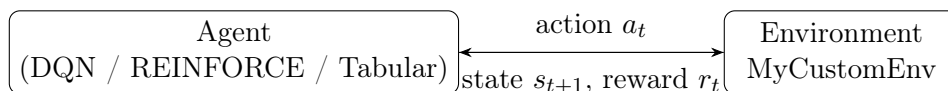


Figura 1: Schema generală agent–mediu folosită în proiect.

2 Environment Design

Mediul este construit pornind de la pachetul `highway_env`, însă logica drumului și a traficului a fost rescrisă în fișierul `custom_env.py`.

Drumul este reprezentat ca un graf de benzi (`RoadNetwork`) și include mai multe secțiuni: un segment drept de intrare, o zonă de îmbinare (*merge*) cu o bandă laterală, un viraj circular, porțiuni drepte suplimentare și o ieșire verticală către un nod final `c0`. Sunt definite mai multe benzi folosind clasele `StraightLane` și `CircularLane`, inclusiv o bandă de intrare laterală pentru vehiculele care se încadrează în trafic.

Vehiculele din trafic sunt de tip `IDMVehicle` (model de conducere IDM) și sunt plasate atât pe secțiuni fixe (obstacole poziționate manual), cât și aleator pe drum. Pentru unele vehicule folosim metoda `plan_route_to` pentru a le forța să urmeze un traseu prin rețea (de exemplu, de la nodul `b0` la `c0`). La fiecare pas de timp, `extthighway_env` actualizează automat dinamica tuturor vehiculelor.

Episodul se termină atunci când vehiculul *ego* se ciocnește sau părăsește drumul, iar trunchierea se produce când timpul depășește `config["duration"]`. Aceste condiții asigură episoade finite și un semnal de învățare bine definit pentru toți agenții.

3 Algoritmi

În același mediu am antrenat și comparat trei tipuri de agenți: un agent tabular (Q-Learning), un agent DQN și un agent de tip policy gradient (REINFORCE cu baseline de valoare).

3.1 Tabular Q-Learning

Agentul tabular folosește o reprezentare discretizată a stării, obținută prin wrapper-ul `extttTabularObsWrapper`. Observația continuă este proiectată într-un vector de 6 componente: poziția *ego* pe o grilă 2D (x, y), un nivel discretizat de combustibil și două variabile discrete care codifică direcția aproximativă către țintă (dx, dy), împreună cu o componentă binară simplă de context. Dimensiunile de discretizare sunt $[6, 6, 3, 2, 3, 3]$, rezultând 1944 de stări posibile.

Pentru fiecare stare discretă se păstrează o valoare $Q(s, a)$ într-un tabel cu 5 acțiuni. Actualizarea este cea clasică de Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a)),$$

cu factor de învățare $\alpha = 0,05$, factor de discount $\gamma = 0,99$ și o strategie ϵ -greedy pentru explorare, cu ϵ care scade treptat de la 1 la 0.05.

3.2 Deep Q-Network (DQN)

Agentul DQN folosește direct vectorul continuu de 25 de dimensiuni ca intrare într-o rețea neuronală feed-forward. Arhitectura implementată în proiect este:

- strat de intrare: vectorul de stare (25 dimensiuni);
- două straturi ascunse dense cu 256 și respectiv 128 de neuroni și activare ReLU;
- strat de ieșire: 5 neuroni care produc valorile $Q(s, a)$ pentru fiecare acțiune discretă.

Rețeaua este antrenată folosind experience replay (buffer de 50 000 de tranziții) și un *target network* sincronizat periodic. La fiecare pas, o tranziție $(s, a, r, s', done)$ este adăugată în buffer, iar când există suficiente exemple se eșantionează mini-loturi de mărime 64. Țintelor de învățare li se aplică fie formula clasică DQN, fie varianta Double DQN (selecție de acțiune cu `policy_net`,

evaluare cu `target_net`). Funcția de pierdere este Huber (`SmoothL1Loss`). Explorarea se face cu o strategie ε -greedy cu ε care scade de la 1.0 la 0.01.

Diagrama de arhitectura DQN este prezentată în Figura 2.

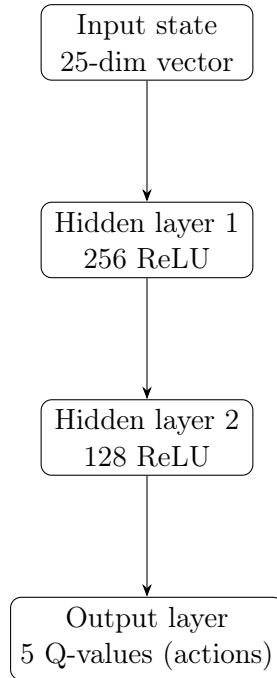


Figura 2: Arhitectura DQN care mapează starea la valorile $Q(s, a)$ pentru cele 5 acțiuni.

3.3 Policy Gradient (REINFORCE + Baseline)

Al treilea agent este unul de tip policy-based. Politica $\pi_\theta(a | s)$ este parametrizată de o rețea neuronală (`PolicyNetwork`) cu un singur strat ascuns de 64 neuroni și ieșire softmax peste cele 5 acțiuni. În plus, am introdus un *value network* care aproximează funcția de valoare $V_\phi(s)$ și este antrenat împreună cu politica.

Update-ul folosește REINFORCE cu advantage:

$$\nabla J(\theta) \approx \sum_t \log \pi_\theta(a_t | s_t) \cdot A_t, \quad A_t = G_t - V_\phi(s_t),$$

unde G_t este returnul Monte Carlo din pasul t . Valoarea este antrenată cu eroare pătratică față de G_t , iar pierderea totală este suma dintre pierderea de politică și o componentă de valoare ponderată. Acest baseline reduce varianța gradientului și îmbunătățește stabilitatea față de REINFORCE simplu.

4 Experimente

Obiectivul experimental a fost compararea celor trei agenți (tabular, DQN și REINFORCE cu baseline) în exact același mediu și cu același set de acțiuni discrete. Pentru fiecare agent am definit un script separat de antrenare (`train_tabular.py`, `train_dqn.py`, `train_reinforce.py`) și am păstrat aceleași condiții de inițializare ale mediului.

Pentru DQN și agentul tabular am rulat câte 500 de episoade de antrenare, în timp ce pentru REINFORCE am folosit 1000 de episoade pentru a compensa varianța mai mare a gradientului de politică. La fiecare episod am înregistrat recompensa cumulată și am salvat un model „best” pe baza mediei mobile a ultimelor 20–50 de episoade.

Pentru a vizualiza evoluția antrenării, scriptul `plot_results.py` încarcă fișierele CSV generate de fiecare agent și desenează curbele de recompensă, împreună cu o versiune netezită. Figura 3 arată clar viteza de convergență și nivelul final de performanță pentru fiecare algoritm.

4.1 Curbe de antrenare

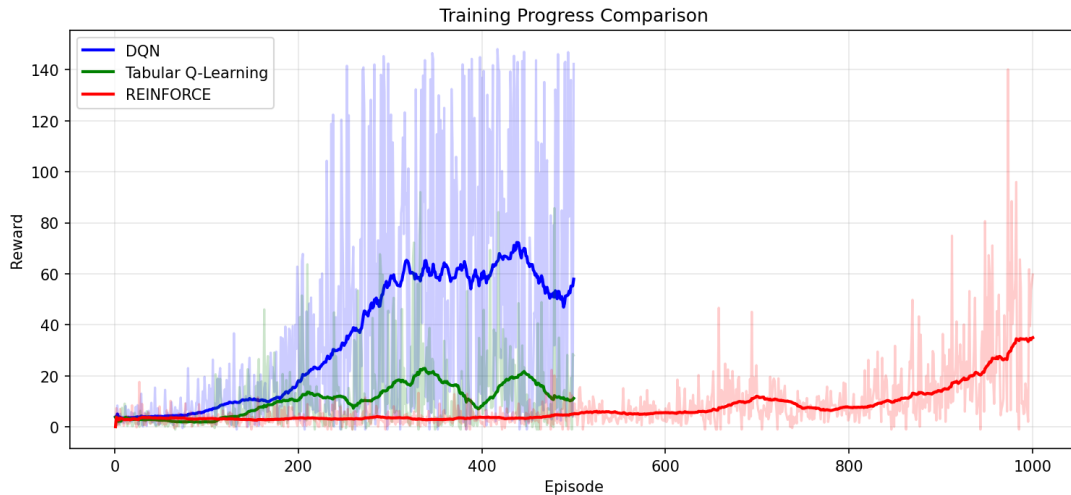


Figura 3: Curbele de recompensă în timpul antrenării pentru toți cei trei agenți.

4.2 Evaluare și comportament

Am implementat și scriptul de evaluare `evaluate_agents.py`, care încarcă modelele antrenate și rulează un număr fix de episoade de test pentru fiecare agent, cu politică pur deterministă (fără explorare, $\epsilon = 0$). Rezultatele sunt salvate în fișiere CSV și reprezentate grafic într-un barplot (`eval_barplot.png`), care permite compararea directă a recompenselor medii.

Comportamentul calitativ al agenților a fost analizat și prin `visualize_actions.py`, care desenează pe hartă pozițiile mașinii *ego*, colorate în funcție de acțiunea aleasă (accelerare, frânare, schimbare de bandă etc.). Aceste hărți de acțiuni ajută la interpretarea tiparelor învățate de fiecare algoritm.

4.3 Hiperparametri și diagrame

Pentru fiecare agent am ales un set de hiperparametri care echilibrează stabilitatea și viteza de învățare:

- **Tabular Q-Learning:** $\alpha = 0,05$, $\gamma = 0,99$, $\epsilon \in [1, 0,05]$ cu decay; rata de învățare relativ mică evită oscilațiile într-un spațiu de stare discret dar dens, iar discount-ul mare pune accent pe recompensele viitoare (traectorii sigure și lungi).
- **DQN:** rata de învățare $5 \cdot 10^{-4}$, $\gamma = 0,99$, buffer de 50 000 tranziții, batch-size 64, ϵ -greedy cu scădere de la 1.0 la 0.01. Un γ mare și replay buffer-ul permit învățarea unor politici mai stabile, iar rețeaua 256–128 controlează un compromis între capacitate și risc de overfitting.
- **REINFORCE:** rata de învățare 10^{-3} , $\gamma = 0,99$, 1000 de episoade. Learning rate-ul moderat și baseline-ul de valoare ajută la reducerea varianței gradientului, dar numărul mai mare de episoade este necesar deoarece update-ul se face la nivel de episod (Monte Carlo).

Diagramele generate sintetizează aceste alegeri:

- **training_comparison.png**: evoluția recompensei pe episod în timpul antrenării pentru toți agenții (curbă brută și netezită);
- **eval_barplot.png**: recompensele medii de evaluare și deviațiile standard, per agent;
- **actions_dqn.png**, **actions_reinforce.png**, **actions_tabular.png**: hărți de acțiuni care arată zonele în care fiecare agent accelerează, frânează sau schimbă banda.

Pentru raport, folosim în mod explicit **training_comparison.png** și **eval_barplot.png** pentru a susține concluziile numerice despre convergență și performanță.

5 Analiză și discuții

Rezultatele numerice arată că agentul DQN obține în medie cele mai mari recompense, atât pe parcursul antrenării, cât și în episoadele de evaluare. Curba sa de învățare crește relativ rapid și se stabilizează la un nivel înalt, semn că rețeaua a învățat o politică robustă de menținere a benzii, evitarea coliziunilor și control al vitezei.

Agentul tabular Q-Learning reușește să învețe un comportament rezonabil, dar rămâne semnificativ mai slab decât DQN. Deși discretizarea de 6 dimensiuni este gestionabilă ca mărime a tabelului, ea pierde informații fine despre dinamica vehiculelor din jur, iar actualizarea prin eșantionare dintr-un mediu complex rămâne dificilă. Totuși, comparativ cu un agent aleator, tabularul îmbunătățește clar frecvența coliziunilor și calitatea traiectoriei.

Agentul REINFORCE cu baseline obține performanțe intermediare: este mai bun decât tabularul, dar în medie mai slab decât DQN. Introducerea unei rețele de valoare a redus varianța și a stabilizat învățarea față de o versiune fără baseline, dar metoda rămâne sensibilă la hiperparametri (rata de învățare, normalizarea avantajelor, lungimea episoadelor). Faptul că folosește doar returnuri Monte Carlo, fără bootstrapping, face ca învățarea să fie mai lentă și mai zgomotoasă.

Vizualizările acțiunilor confirmă aceste concluzii: DQN tinde să păstreze o traiectorie mai fluidă, cu schimbări de bandă planificate și viteză constantă, în timp ce REINFORCE afișează uneori oscilații între accelerare și frânare. Agentul tabular ia decizii mai „rigide”, fiind limitat de discretizarea mai grosieră a stării.

6 Concluzii și direcții viitoare

În acest proiect am definit un mediu personalizat de conducere pe baza `extthighway_env` și am comparat trei abordări de învățare prin întărire: Q-Learning tabular, Deep Q-Network și REINFORCE cu baseline. Rezultatele experimentale arată că, pentru o reprezentare de stare continuă și relativ bogată (25 de dimensiuni), metodele bazate pe funcții de aproximare profunde (DQN) oferă un avantaj clar față de un tabel discret simplu. În același timp, un agent policy-based cu baseline poate atinge performanțe rezonabile, dar necesită o reglare mai atentă a hiperparametrilor.

Ca direcții viitoare, proiectul poate fi extins în mai multe moduri:

- trecerea la algoritmi actor-critic mai avansați (PPO, A2C), care combină avantajele bootstrapping-ului cu stabilitatea unui obiectiv cu constrângeri;
- îmbunătățirea funcției de recompensă și a configurației de trafic, pentru a viza scenarii mai realiste (depașiri, intersecții, limitări de viteză variabile);
- analizarea robusteții politicilor învățate la variații ale dinamicii vehiculelor sau ale senzorilor (de exemplu zgomot pe observații);
- integrarea proiectului într-o interfață grafică interactivă pentru demonstrații în timp real.

Per ansamblu, proiectul demonstrează modul în care conceptele teoretice de RL (stare, acțiune, recompensă, Q-Learning, DQN, policy gradient) pot fi aplicate într-un scenariu de conducere autonomă simplificat, respectând cerințele de implementare, experimentare și documentare ale cursului.